

Lesson 6 ROS2 Package

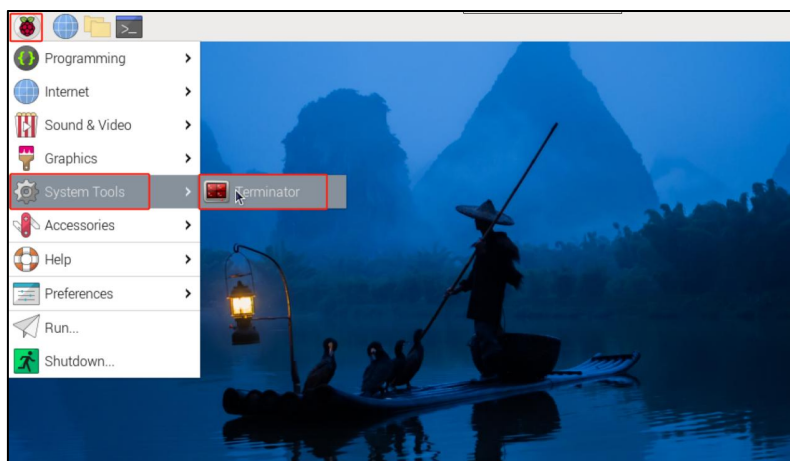
1. Package Introduction

Each robot may have many functionalities, such as motion control, visual perception, autonomous navigation, etc. It's possible to put the source code of these functions together, what if we want to share some of these functions with others? It can be found that the code is all mixed up and difficult to separate.

The principle of packages addresses this issue. The code of different function is divided into different packages, aiming to reduce their coupling as much as possible. When we need to share some functions with others in the ROS community, we only need to explain how to use the package, and others can quickly use it. Therefore, the mechanism of packages is one of the important methods to improve software reusability in ROS.

2. Create and Compile Package

1) Click  in the upper-left corner, then select “**System Tools and Terminator**” in sequence.



2) Enter the command “**cd hiwonder_ws/src/**” to switch to the src folder within the workspace named **hiwonder_ws**.

```
ubuntu@raspberrypi:~$ cd hiwonder_ws/src/  
ubuntu@raspberrypi:~/hiwonder_ws/src$
```

3) Enter “**ros2 pkg create hello_world_demo --build-type ament_python --dependencies rclpy --node-name hello_world**” and press Enter to create a package named **hello_world_demo**. Add the dependency “**rclpy**” and generate an executable program named “**hello_world**”.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ ros2 pkg create hello_world_demo --build-type ament_python --dependencies rclpy --node-name hello_world
```

4) Enter “**cd ~/hiwonder_ws**” to switch to the root directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src$ cd ~/hiwonder_ws  
ubuntu@raspberrypi:~/hiwonder_ws$
```

5) Enter “**colcon build**” to compile packages in workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws$ colcon build
```

6) After successful compilation, enter “**cd src/hello_world_demo/**” to enter the directory where the package is located, and press Enter to verify whether the package is created successfully.

```
ubuntu@raspberrypi:~/hiwonder_ws$ cd src/hello_world_demo/  
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$
```

3. Package Instruction

After compilation, enter the command “**tree -L 1**” to view the root directory of the workspace.

```
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$ tree -L 1
.
├── hello_world_demo
├── package.xml
├── resource
├── setup.cfg
├── setup.py
└── test

3 directories, 3 files
ubuntu@raspberrypi:~/hiwonder_ws/src/hello_world_demo$
```

In ROS system, the structure of a typical package is shown above. The “hello_world_demo” is the root directory of the package, there will be 6 files or subdirectories inside.

Name	Instruction
hello_world_demo	Package directory: Python source file directory
package.xml	Package information, such as package name, version, author, dependencies.
resource	Resource directory
setup.cfg	Package basic configuration file
setup.py	Similar to CMakeLists.txt file for C++ packages
test	Store test-related files